

클라우드 네이티브 엣지 환경을 위한 서버리스 WebAssembly 성능 최적화 및 런타임 비교 분석

Performance Optimization of Serverless WebAssembly in Cloud-Native Edge Environments: Comparative Analysis of Container, Python Wasm, and Rust Wasm Runtimes

고동현
Ko DongHyeon

동국대학교 공과대학 컴퓨터정보통신공학부 컴퓨터공학전공
Email : kodh0204@gmail.com

요약

클라우드 네이티브 환경이 데이터센터를 넘어 IoT 및 엣지(Edge)로 확장됨에 따라, 제한된 리소스 내에서 대용량 데이터를 실시간으로 처리해야 하는 기술적 난제에 직면하고 있다. 기존의 Docker 컨테이너 기술은 높은 이식성을 제공하지만, 무거운 이미지 크기와 느린 콜드 스타트 간으로 인해 순간적인 트래픽 폭주에 대응하는 데 한계를 보인다.

본 연구는 이에 대한 대안으로 Kubernetes 기반의 WebAssembly(Wasm) 런타임인 SpinKube를 도입하고, 2단계에 걸친 성능 검증 및 최적화를 수행하였다. 1단계 연구에서는 Docker 컨테이너와 Python 기반 Wasm을 비교하여, Wasm이 시작 속도(10배)와 집적도(5배) 면에서 우수함을 입증하였다. 2단계 연구에서는 Python 런타임의 연산 속도 한계를 극복하기 위해 핵심 로직을 Rust(Native Wasm)로 마이그레이션하였다. 그 결과, 바이너리 크기를 91배 감소(0.39MB)시키고 처리량을 7.8배 향상시켜 프로덕션 레벨의 성능을 확보하였다.

본 연구는 정량적 데이터를 통해 Rust 기반의 WebAssembly가 엣지 컴퓨팅을 위한 최적의 서버리스 모델임을 제안하며, 향후 Azure AKS 환경에서의 확장 전략을 제시한다.

1. 서론

1.1 연구 배경

IoT 센서 데이터 수집과 같은 워크로드는 트래픽이 간헐적이고 예측 불가능하다는 특징을 가진다. 이러한 환경에서는 요청이 없을 때 리소스를 0으로 줄이고 (Scale-to-Zero), 요청 발생 시 밀리초 단위로 확장 (Scale-out)하는 Instant Serverless 아키텍처가 필수적이다. [1],[2]

1.2 문제 정의 및 기존 도구의 한계

기존 컨테이너(Docker) 방식은 OS 가상화를 포함하므로 애플리케이션 구동까지 수 초가 소요되어 실시간 데이터 처리에 지연을 발생시킨다. 이를 해결하기 위해 WebAssembly(Wasm)가 도입되었으나, 초기 프로토타입에 사용된 Python(componentize-py) 런타임은 인터프리터 오버헤드로 인해 순수 연산 성능과 바이너리 크기 면에서 최적화의 여지를 남겼다.

1.3 연구 목적

본 연구의 목표는 다음과 같다.

1. 아키텍처 검증: SpinKube(Wasm)가 Docker 대비 엣지 환경에 적합한지 정량적으로 검증한다.
2. 런타임 최적화: Python Wasm의 한계를 분석하고, Rust 언어로 마이그레이션하여 극한의 성능 효율을 달성한다.
3. 비용 효율성 입증: 고밀도 집적을 통해 클라우드 인프라 비용 절감 효과를 증명한다.

2. 관련 연구 및 기술

2.1 SpinKube & Kubernetes

SpinKube는 Kubernetes 환경에서 Wasm 애플리케이션을 파드(Pod)처럼 관리하는 오픈소스 프로젝트다. 이는 containerd-shim-spin을 통해 Docker 컨테이너 런타임을 우회하여 Wasm 모듈을 직접 실행함으로써 오버헤드를 최소화한다.

2.2 Scale-to-Zero와 KEDA

KEDA(Kubernetes Event-driven Autoscaling)는 이벤트 소스(ex: AWS Kinesis, Kafka)의 메트릭을 기반으로 파드의 개수를 0에서 N까지 자동으로 조절한다. 이 과정에서 0에서 1로 확장되는 '콜드 스타트' 속도가 시스템 전체의 성능을 좌우한다.

3. WASBM(Python) vs Container 성능 비교

3.1 실험 설계

동일한 IoT 로그 분석 로직(에러 감지 및 응답 시간 분석)을 Docker(Flask/Python)와 SpinKube(Spin/Python) 환경에 각각 구현하여 비교하였다.

- 대조군 (Container): 128Mi 메모리 할당 (OS 오버헤드 고려)
- 실험군 (SpinKube): 16Mi 메모리 할당 (경량성 고려)

3.2 실험 결과

비교 항목	Docker Container	SpinKube (Python)	결과 분석
시작 속도 (Cold Start)	10,762ms (~10.7초)	1,037ms (~1.0초)	10.4배 빠름
이미지 크기	167 MB	15 MB	91% 절감
집적도	10 Pods / Node	50 Pods / Node	5.0배 향상
연산 속도 (평균)	1.6ms	8.0ms	Container 우위 (4.9배)

표 1: Python 환경에서 WASM vs Container 성능 비교 3000건의 IoT 시뮬레이션 요청

3.3 소결

SpinKube는 시작 속도와 리소스 효율성에서 압도적인 우위를 보였다. 그러나 연산 속도 측면에서 네이티브 Python(Docker)보다 Wasm Python이 약 5배 느린 것으로 나타났다. 이는 Wasm 내부에서 파이썬 인터프리터가 구동되는 구조적 한계 때문으로 분석되었다.

4. Rust 마이그레이션을 통한 성능 최적화

4.1 최적화 배경

앞선 실험에서 확인된 연산 속도 저하와 15MB라는 Wasm 치고는 큰 바이너리 크기를 해결하기 위해, 런타임 언어를 Python에서 Rust로 전면 교체하기로 결정하였다. Rust는 가비지 컬렉터(GC)가 없고 Wasm으로 직접 컴파일(Native Compilation)되므로 오버헤드가 없다.

4.2 실험 설계

- Legacy: Python Code + CPython Interpreter + Standard Lib → Wasm
- Target: Rust Code → wasm32-wasip1 Target Compile

4.3 최적화 결과

메트릭	Python (Wasm)	Rust (Native)	개선 효과
Binary Size	35.86 MB	0.39 MB	91배 감소
Startup + Avg Latency	~18.09 ms	~2.23 ms	8.1배 빨라짐
Throughput	~1,713 req/s	~13,389 req/s	7.8배 증가
P95 Latency	31.18 ms	3.84 ms	8.1배 더 안정적

표 2: WASM Python vs Rust 성능 비교 10000건의 IoT 시뮬레이션 요청

4.3.1 바이너리 크기 (Binary Size)

- Python: 35.86 MB (라이브러리 포함 시 증가)
- Rust: 0.39 MB

91배의 크기 감소를 달성했다. 0.39MB는 네트워크 상태가 열악한 엣지 디바이스에서도 즉시 전송 가능한 크기다.

4.3.2 지연 시간 (Latency)

- Python: ~18.09 ms (Startup 포함)
- Rust: ~2.23 ms

8.1배 빨라졌다. 인터프리터 초기화 과정이 사라지면서, 요청 도달 즉시 비즈니스 로직이 실행됨을 확인했다.

4.3.3 처리량 (Throughput)

- Python: ~1,713 req/s
- Rust: ~13,389 req/s

7.8배 증가했다. 이는 동일한 서버 자원으로 8배 더 많은 트래픽을 처리할 수 있음을 의미한다.

5. 논의 및 비즈니스 임팩트

5.1 비용 효율성 시뮬레이션

Azure AKS 환경에서 1,000개의 워크로드를 처리한다고 가정했을 때의 월간 비용을 시뮬레이션하였다.

- Docker 기반: \$1,458
- SpinKube (Rust) 기반: \$565

Rust 기반 Wasm 도입 시, 컴퓨팅 및 네트워크 비용을 포함하여 약 65%의 비용 절감이 가능하다.

5.2 기술 선택 가이드라인

본 연구 결과를 바탕으로 다음과 같은 기술 도입 기준을 제안한다.

- Python Wasm: 빠른 프로토타이핑이나 데이터 과학 라이브러리가 필요한 경우.
- Rust Wasm: 프로덕션 환경, 고성능이 요구되는 핫패스, 리소스가 극도로 제한된 초소형 엣지 디바이스.

6. 향후 계획

본 연구는 로컬 시뮬레이션 환경에서의 검증을 마쳤으며, 향후 Microsoft Azure 클라우드 환경으로 확장할 계획이다.

6.1 Azure Kubernetes Service (AKS) 도입

Azure는 현재 Wasm Node Pools 기능을 프리뷰로 제공하며 업계를 선도하고 있다. 본 시스템을 AKS에 배포하여 관리형 쿠버네티스 환경에서의 안정성을 검증할 것이다.

6.2 대규모 확장성 테스트

AKS 환경에서 가상의 IoT 디바이스 10만 대를 모사하여, Azure Event Hubs와 연동된 대규모 오토스케일링 성능을 테스트하고, Azure Monitor를 통해 엔드투엔드 가시성을 확보할 예정이다.

7. 결 론

본 연구를 통해 SpinKube와 Rust의 결합은 기존 컨테이너 기술의 한계를 뛰어넘는 차세대 엣지 컴퓨팅 솔루션임을 입증하였다.

- Docker 대비: 10배 빠른 기동성과 5배 높은 집적도.
- Python Wasm 대비: 91배 작은 크기와 8배 높은 처리량.

이러한 결과는 Rust로 작성된 WebAssembly가 미래의 클라우드 네이티브 엣지 환경에서 표준 런타임(De Facto Standard)이 될 잠재력이 충분함을 시사한다.

8. 참고문헌

1. Gadepalli, P. K., et al. "Sledge: a Serverless-first, Light-weight Wasm Runtime for the Edge." Middleware '20.
2. Fermyon. "Spin Up Hub: Language Support Guide & Benchmarks." 2024.
3. Microsoft. "AKS Wasm Node Pools Preview Documentation." Microsoft Learn.